

Web Programming I
Academic Year 2025-2026
Project

PROJECT TITLE

Adopt Don't Shop :Pet Adoption & Rescue Web Application

TEAM

First name: Maria

Last name: Elia

e-mail: 202211515@ua.edu.lb

First name: Serena

Last name: Aouad

e-mail: 202210400@ua.edu.lb

DESCRIPTION OF THE PROJECT

This project aims to develop a user-centric pet adoption web application that connects animal shelters and individuals seeking to adopt pets. The platform will provide an intuitive interface for browsing pets, submitting adoption requests, and managing shelter listings. By leveraging a multitier software architecture with a Node.js backend and a modern frontend framework, the application will ensure scalability, security, and a seamless user experience. The system will incorporate database integration, modular design, and secure APIs to facilitate smooth interactions between adopters, shelters, and administrators.

MAIN GOALS

- Build a responsive and interactive web application for pet adoption.
- Ensure secure authentication and role-based access(adopters, shelter).
- Design and implement a scalable database schema to manage users, pets, and adoption requests.
- Develop RESTful APIs to support backend–frontend communication.
- Provide intuitive and user-friendly interfaces for all roles.
- Incorporate security measures to protect user data and prevent unauthorized access.
- Deliver a fully tested, documented, and deployable web solution.
- Implement automated notifications (email) for adoption request approvals.

CORE FEATURES

1- Authentication & Authorization

- Secure login and registration with password encryption.
- Role-based access:
 - **Adopters** → register, browse pets, request adoption.
 - **Shelters** → add pets, manage adoption requests.

2- Pet Profiles

- Name, species, breed, age, gender.
- Photo gallery.
- Status: *Available*, *Adopted*.

3- Search & Filter

- Search pets by name.
- Filter pets by species, breed, gender.
- Sort pets by name, age.

4- Adoption Request System

- Users submit adoption requests.
- Shelters approve/reject requests.
- Request history and status tracking.

5- Notifications

- Email notifications for adoption request approval.

TECHNOLOGY STACK AND SYSTEM COMPONENTS

1-Frontend Technology:

Chosen Tool: HTML AND CSS

- HTML, CSS AND JAVASCRIPT: For structure, styling and interactivity.

We choose this frontend technology, so users can interact smoothly with the platform (search pets, view profiles, fill adoption forms).

2-Database Selection:

Chosen Tool : MySQL

- MySQL: Relational database supports primary/foreign keys to model pet-user relationships.

We choose this database to store structured data like pet profiles, adoption requests and user accounts .

3-Authentication and Authorization:

Chosen tools: JWT-based authentication.

Role-based access control:

- Adopters: create requests, view their requests, see adopted pets.
- Shelter: approve/reject requests, view all requests, add/edit pets.

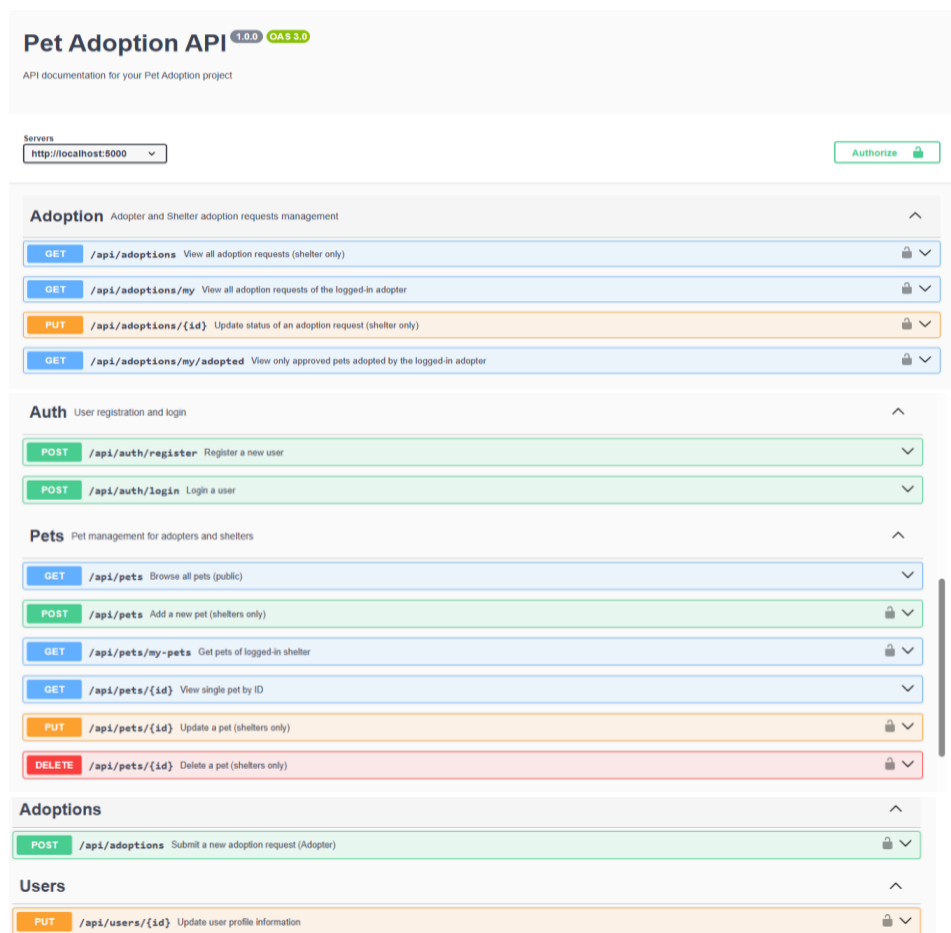
We choose it for authentication and authorization, so users can register, log in and apply for adoption and the shelter can manage pets, approve/reject adoption requests.

4-APIs and Communication:

Chosen Tools: RESTful APIs (Express.js) and OpenAPI/Swagger

- REST APIs: They are the industry standard for communication between frontend (React) and backend (Node.js).
- Swagger: Ensures proper documentation of all endpoints.

We choose RESTful APIs and Swagger because they enable communication between frontend (React) and backend (Node.js) like fetching pets list, submit adoption requests, authenticate users.



```
{
  "success": true,
  "data": [
    {
      "pet_id": 3,
      "name": "Bella",
      "species": "cat",
      "breed":
        "american shorthair",
      "age": 4,
      "gender":
        "female",
      "status":
        "adopted",
      "photo_url":
        "http://localhost:5000/
        uploads/176306763978
        4.jpeg",
      "user_id": 6
    }
  ]
}
```

DESIGN AND DEVELOPMENT OF SOFTWARE MODULES

System Overview

The Pet Adoption & Rescue Web Application is organized into modular components that handle specific functionality areas. Each module contains its own routes, controllers, model, and validation logic, ensuring a clean separation of concerns and scalability.

Software modules and responsibilities

1-Authentication Module: Handles user registration, login, and role-based access control.

JWT authentication, password verification.

➤ Files: authMiddleware.js.

2-Users Module: Handles registration, login, and user management.

➤ Files: userModel.js, userController.js, userRoutes.js.

3-Pets Module: CRUD operations for pet profiles.

➤ Files: petModel.js, petController.js, petRoutes.js.

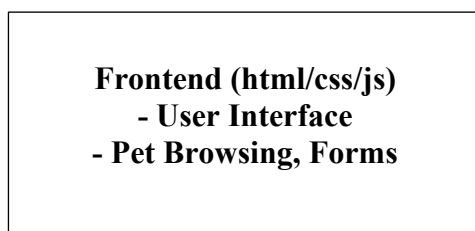
4-Adoption Module: Manage adoption requests, status updates, and email notifications.

➤ Files: adoptionModel.js, adoptionController.js, adoptionRoutes.js.

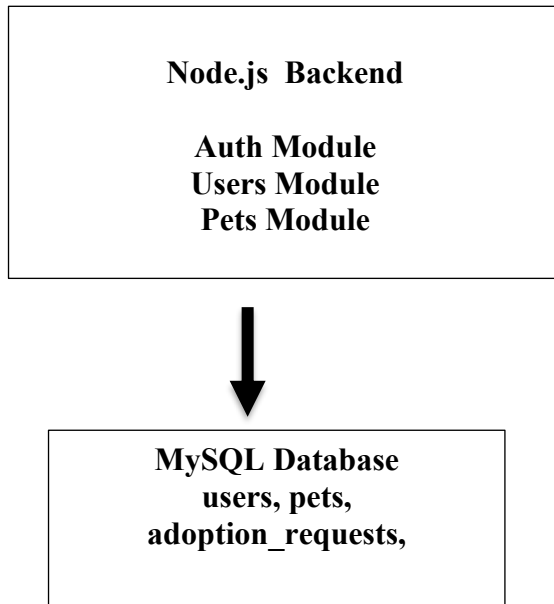
Module Architecture

module/
├ routes/ → API endpoints (Express.js)
├ controllers/ → Handle requests & responses
├ models/ → interact with database via queries.
├ validations/ → ensure input is correct and safe (e.g., required fields)
└ tests/ → ensure correctness of model functions and endpoints.

System Architecture Diagram



RESTful APIs



Database Schema

TABLE	KEY FIELDS	RELATIONSHIPS
USERS	user_id (PK), name, email, password, role	1-* with AdoptionRequests, 1-* with Pets (if shelter)
PETS	pet_id (PK), name, species, breed, age, gender, status, photo_url, user_id (FK)	*-1 with Shelter (User), - with AdoptionRequests
ADOPTIONREQUESTS	request_id (PK), pet_id (FK), adopter_id (FK), fullName, phone, message, status, request_date	*-1 with Pets, *-1 with Users

Relationships:

- A User (Adopter) can submit multiple AdoptionRequests.
- A Shelter (User) can list multiple Pets.
- An Admin oversees Shelters and manages Users.

API Documentation & Communication

ENDPOINT	METHOD	DESCRIPTION	INPUT	OUTPUT
/API/PETS	GET	Get all pets (supports pagination, filtering, sorting)	query params	List of pets
/API/PETS/:ID	GET	Get pet details	pet_id	Pet object
/API/PETS	POST	Add new pet	JSON body	Confirmation message
/API/PETS/:ID	PUT	Update pet info	JSON body	Updated record
/API/PETS/:ID	DELETE	Delete pet	pet_id	Success message

Validation and Error Handling

- Backend checks for missing required fields, invalid email format, unauthorized access.
- Sends clear JSON error responses to frontend.

Example:

```
app.use((err, req, res, next) => {  
  
  res.status(err.status || 500).json({  
  
    success: false,  
  
    message: err.message || "Internal Server Error"  
  
  });  
  
});
```

External Integrations

Email Service using nodemailer to notify adopters when their request is approved.

Integration and Testing

Integration:

- All backend modules (Users, Auth, Pets, Adoptions) were integrated with the frontend using RESTful API endpoints.
- JWT authentication was implemented across all protected routes to secure access based on user roles.
- Email notifications were integrated using nodemailer to inform adopters about the approval of their adoption requests.
- Frontend fetches data from backend endpoints with proper headers and handles responses to update the UI in real-time.
- Dependency flow:
 - Adopter submits request → Adoption module saves it → Shelter views requests → Approves request → Pet status updated → Email sent to adopter.

Testing:

- **Manual Testing:**
 - Verified that users can sign up, log in, and view their dashboard correctly.
 - Created adoption requests and ensured they appear in the shelter dashboard.
 - Tested approval/rejection workflow for requests.
 - Verified email notifications are sent upon approval.
 - Checked access restrictions: adopters cannot access shelter endpoints and vice versa.
- **Automated Tests :**
 - Unit tests for model functions (e.g., `getUserById`, `createAdoptionRequest`).
 - API endpoint tests using tools like Postman or Jest to validate expected responses.

Error Handling and Validation Tests:

- Attempted requests with missing required fields → Correct 400 responses received.
- Unauthorized requests without JWT → Correct 401 responses received.
- Invalid request updates (e.g., approving a non-existent adoption request) → Correct 404 response returned.

Pagination/Filtering/Sorting Testing:

- **Pagination:**

GET `/api/pets?limit=10&offset=20` retrieves the next page of results (pets 21–30).
Used for efficient browsing and performance optimization.
- **Filtering:**

Example: GET `/api/pets?species=dog&maxAge=3`
Returns only dogs younger than 3 years old.
Helps adopters quickly find relevant pets.
- **Sorting:**

GET `/api/pets?sort=name` sorts the pet list alphabetically.
Allows adopters to browse pets in a clean and predictable order.

External Integration Testing:

- Tested nodemailer integration by sending approval emails to multiple adopter accounts.
- Verified email content, formatting, and correct recipient addresses.
- Simulated network errors to ensure backend logs errors without crashing.

Screenshots:

- User dashboards

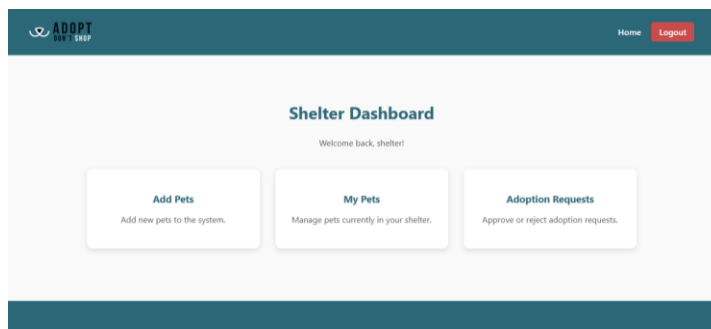


Figure 1 _shelter dashboard

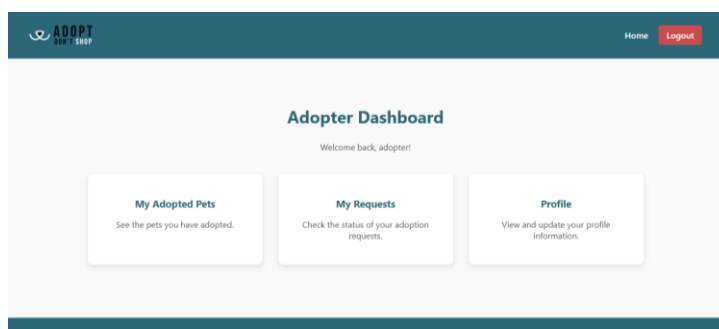


Figure 2 _adopter dashboard

- Pet listing page with search/filter

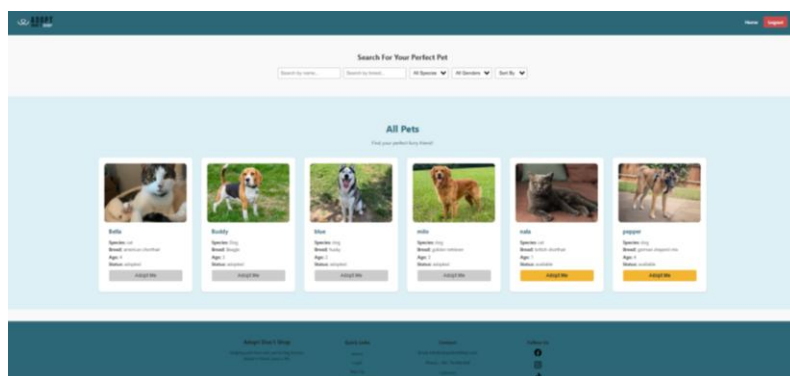


Figure 3 _pets page

- Adoption request page

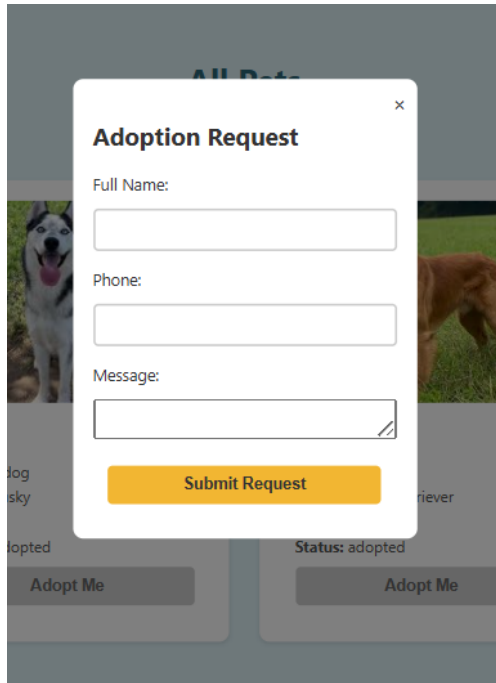
A screenshot of a web application showing an 'Adoption Request' form overlay. The form is white with a black border and a close button (X) in the top right corner. It contains three input fields: 'Full Name:', 'Phone:', and 'Message:'. Below these fields is a yellow 'Submit Request' button. The background is a blurred image of a dog and a cat, with some text like 'Status: adopted' and 'Adopt Me' visible.

Figure 4 _ adoption request

- Email notifications received.

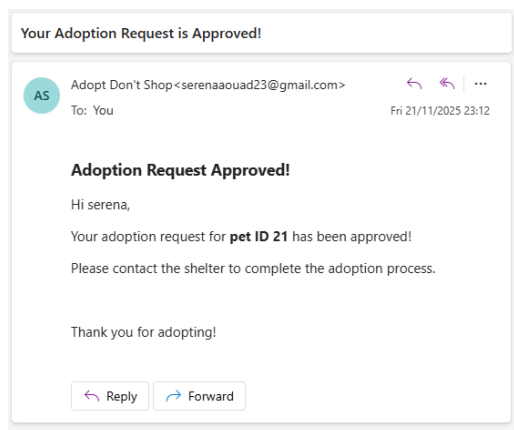


Figure 5 _email verification